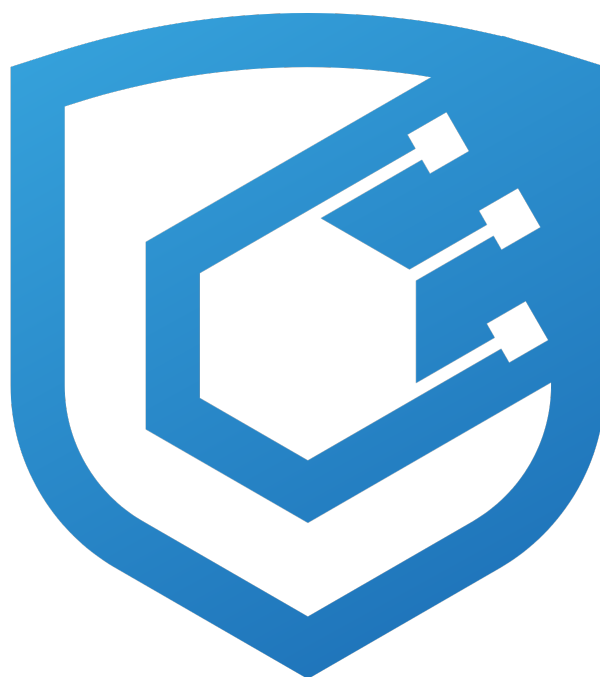




Accountable PR75 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[Alix40](#)

June 30, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	4
6.1	Centralization Risks	4
6.2	Defensive Improvements	4
7	Findings	7
7.1	Medium Risk	7
7.1.1	DepositGateway::settle prices escrowed deposits at the live share price, not the epoch's closing NAV	7
7.1.2	DepositGateway::cancelDeposit is a costless NAV-direction option that lets a depositor front-run AccountableYield::publishRate to avoid unfavorable pricing	8
7.1.3	AccountableYield lacks an InDefaultClaims wind-down and permanently strands unfulfilled redemptions after a default	9
7.1.4	AccountableStrategy::acceptDefault has no loan-state guard, letting a fully repaid loan be forced into InDefault and bricking the vault	10
7.2	Low Risk	13
7.2.1	DepositGateway::settle, refund leave escrow stuck when the strategy is stale or paused and _refundable recognizes neither, permanently inflating unsettledEscrow	13
7.2.2	AccountableYield::updateLateStatus is permissionless and lacks the whenNotStale and whenNotPaused guards	13
7.2.3	DepositGateway::forcePassEpoch is permissionless and can strip a pending depositor's cancel window	14
7.2.4	DepositGateway::requestDeposit minDeposit gate is weaker than the vault's MIN_AMOUNT_WEI floor, leaving sub-floor requests permanently unseizable and non-refundable	14
7.2.5	AccountableYield::_accruedFeeShares degenerate fee-overflow early return advances the high-water mark without charging the fee	15
7.2.6	DepositGateway::requestDeposit capacity reservation lets one requester exhaust headroom and block all other depositors	16
7.2.7	AccountableYield::acceptTerms writes new penalty parameters before accruing, mispricing the proposal-to-acceptance window	16
7.3	Informational	18
7.3.1	DepositGateway::requestDeposit omits the constructor's KYC rejection on a later Whitelist-to-KYC flip	18
7.3.2	DepositGatewayFactory::createDepositGateway is permissionless and registry-less, creating a poisoned-gateway social-engineering surface	18
7.3.3	AccountableYield::onMint entry path is permanently dead once a deposit gateway is wired because the gateway only calls vault.deposit	19
7.3.4	AccountableYield still writes lostAssets and lastTotalAssets but no on-chain logic reads either after PR75 rewrote the fee path to a per-share HWM	19
7.3.5	DepositGateway::_refundable carries an inverted minDeposit comment that misstates the setter-regression direction	20
7.3.6	Missing or incomplete events: absent emissions, missing indexed params and missing event fields	20
7.3.7	DepositGatewayFactory::createDepositGateway has a dead gateway == address(0) check after new	21

7.3.8	DepositGateway::_passEpoch passes an emptied epoch into Passed where it never finalizes to Settled	21
7.3.9	DepositGateway wraps vault IAccess reads in redundant try/catch guards that cannot fail for a valid vault	22
7.3.10	Redemptions are priced at the live NAV with no front-running protection, unlike deposits, letting a holder exit ahead of a NAV-loss publish	22
7.3.11	AccountableYield::_accruePenalties includes accruedPenalties in the penalty base, charging compound interest instead of simple interest	24
7.4	Gas Optimization	25
7.4.1	DepositGateway::_passEpoch re-derives the epochs[epochId] storage pointer the callers already hold	25
7.4.2	DepositGateway::_passEpoch re-reads strategy NAV every caller already fetched	25
7.4.3	AccountableYield::_accrueFees re-reads the vault totalSupply already loaded by _accruedFeeShares	25

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Accountable is a net-asset-value (NAV) based yield and credit strategy. `AccountableYield` is a UUPS-upgradeable strategy attached to an asynchronous ERC4626-style vault: users deposit an arbitrary per-instance ERC20 at the current NAV-derived share price, a borrower draws the deposited capital, and a trusted `dvnPublisher` role periodically reports the deployed value through `publishRate`, which drives the share price and accrues performance and management fees as newly minted shares. Redemptions are asynchronous through a withdrawal queue. The system is deployed as per-strategy proxies across Ethereum Mainnet, Arbitrum One, Citrea, Monad, and Bob.

PR75 (“deposit gateway”) adds an opt-in batching layer in front of the strategy’s synchronous deposit path. A permissionless `DepositGatewayFactory` deploys a `DepositGateway` bound to one strategy; once the manager wires it via `setDepositGateway`, the strategy rejects any deposit controller other than the gateway, forcing retail deposits to commit before the next NAV is published. Depositors escrow assets through `requestDeposit`; an epoch closes when the strategy publishes a new NAV (or via permissionless `forcePassEpoch`), after which a permissionless `settle` mints vault shares for each escrowed request and `refund/cancelDeposit` return escrow. PR75 also reworks the performance-fee high-water mark from an absolute-asset peak to a per-share-price ratio (`peakSharePrice`) and adds a `reinitialize` migration hook to seed it.

5 Audit Scope

The audit scope was limited to the contents of [PR75](#):

```
// new
src/factory/DepositGatewayFactory.sol
src/modules/DepositGateway.sol

// added extra fields
src/strategies/storage/YieldStorage.sol

// some changes
src/strategies/AccountableYield.sol
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [PR75](#) code provided by [Accountable](#). The findings consist of 4 Medium & 7 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The protocol relies on several privileged roles, whose capabilities are described below.

The investment manager wires the deposit gateway via `AccountableYield::setDepositGateway`. Once set, only that gateway can deposit, and it cannot be reset to the zero address, so returning to direct deposits requires an upgrade. If the gateway is re-pointed while the previous one still holds a passed-but-unsettled epoch, that epoch's escrow is recoverable only via `refund`.

The investment manager (or security admin, when enabled) can call `reinitialize`, a one-shot hook that resets the fee high-water mark `peakSharePrice` from the live price and emits `PeakSharePriceReset`. Because it does not accrue pending fees first, the reset shifts the basis for subsequent performance fees. It exists only in the upgrade implementation for the existing vault, not in factory-deployed strategies.

Gateway deployment and wiring are manual and unscripted, so correct setup depends on operator discipline.

The `dvnPublisher` role publishes the NAV that prices deposits and fee accrual. Per [Accountable's](#) design this is a two-step, consensus flow: the value comes from [Accountable's](#) Data Verification Network, is reconciled against an independent shadow-NAV (three sources must converge), then submitted on-chain by a manager or manager-delegated executor, with `publishRate` rejecting stale or out-of-order measurements; the address is wired by the operations admin, not the manager - so the role submits a pre-validated value rather than an arbitrary one. NAV correctness and timeliness rest on that pipeline: a stale or absent NAV blocks settlement and shapes the price escrowed depositors realize.

6.2 Defensive Improvements

Beyond the filed findings, these defense-in-depth improvements are worth considering:

- Settlement depends on operators settling passed epochs promptly; an explicit settle deadline (or a per-share price frozen at epoch close) would remove the reliance on keeper timing.
- Run the implementation switch and the `reinitialize` migration atomically on every `AccountableYield` proxy (e.g. via `upgradeToAndCall`). Because the `peakSharePrice` slot changes units (absolute amount to PRECISION-scaled ratio), a two-step upgrade that lets fees accrue before `reinitialize` would charge against a stale mark. The team has confirmed atomic upgrades; committing the scripts makes this auditable, and gating fee accrual behind a migration flag until `reinitialize` runs adds defense in depth.
- Fee-affecting migration and configuration entry points should emit both old and new values (`reinitialize` currently emits only the new `peakSharePrice`), so off-chain monitoring can tie anomalies to the action behind them.
- `DepositGateway::settle`'s bare `catch {}` cannot distinguish a genuinely undeliverable request from an out-of-gas revert (via the 63/64 gas rule, a starved inner `deposit` reverts while `settle` survives, silently skipping the request). No funds are at risk - the catch mutates no state and settlement is retryable - so as hardening, assert a minimum `gasleft()` before the call and narrow the catch to the expected revert.

Summary

Project Name	PR75
Repository	credit-vaults-internal
Commit	69f74ade0441...
Fix Commit	fcc871a3fca7...
Audit Timeline	June 23th - June 26th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	4
Low Risk	7
Informational	11
Gas Optimizations	3
Total Issues	25

Summary of Findings

[M-1] DepositGateway::settle prices escrowed deposits at the live share price, not the epoch's closing NAV	Resolved
[M-2] DepositGateway::cancelDeposit is a costless NAV-direction option that lets a depositor front-run AccountableYield::publishRate to avoid unfavorable pricing	Resolved
[M-3] AccountableYield lacks an InDefaultClaims wind-down and permanently strands unfulfilled redemptions after a default	Resolved
[M-4] AccountableStrategy::acceptDefault has no loan-state guard, letting a fully repaid loan be forced into InDefault and bricking the vault	Resolved
[L-1] DepositGateway::settle, refund leave escrow stuck when the strategy is stale or paused and _refundable recognizes neither, permanently inflating unsettledEscrow	Resolved
[L-2] AccountableYield::updateLateStatus is permissionless and lacks the whenNotStale and whenNotPaused guards	Resolved
[L-3] DepositGateway::forcePassEpoch is permissionless and can strip a pending depositor's cancel window	Resolved
[L-4] DepositGateway::requestDeposit minDeposit gate is weaker than the vault's MIN_AMOUNT_WEI floor, leaving sub-floor requests permanently unsettleable and non-refundable	Resolved
[L-5] AccountableYield::_accruedFeeShares degenerate fee-overflow early return advances the high-water mark without charging the fee	Resolved

[L-6] <code>DepositGateway::requestDeposit</code> capacity reservation lets one requester exhaust headroom and block all other depositors	Resolved
[L-7] <code>AccountableYield::acceptTerms</code> writes new penalty parameters before accruing, mispricing the proposal-to-acceptance window	Resolved
[I-01] <code>DepositGateway::requestDeposit</code> omits the constructor's KYC rejection on a later Whitelist-to-KYC flip	Resolved
[I-02] <code>DepositGatewayFactory::createDepositGateway</code> is permissionless and registry-less, creating a poisoned-gateway social-engineering surface	Resolved
[I-03] <code>AccountableYield::onMint</code> entry path is permanently dead once a deposit gateway is wired because the gateway only calls <code>vault.deposit</code>	Resolved
[I-04] <code>AccountableYield</code> still writes <code>lostAssets</code> and <code>lastTotalAssets</code> but no on-chain logic reads either after PR75 rewrote the fee path to a per-share HWM	Resolved
[I-05] <code>DepositGateway::_refundable</code> carries an inverted <code>minDeposit</code> comment that misstates the setter-regression direction	Resolved
[I-06] Missing or incomplete events: absent emissions, missing indexed params and missing event fields	Resolved
[I-07] <code>DepositGatewayFactory::createDepositGateway</code> has a dead gateway <code>== address(0)</code> check after new	Resolved
[I-08] <code>DepositGateway::_passEpoch</code> passes an emptied epoch into <code>Passed</code> where it never finalizes to <code>Settled</code>	Resolved
[I-09] <code>DepositGateway</code> wraps <code>vault IAccess</code> reads in redundant <code>try/catch</code> guards that cannot fail for a valid vault	Resolved
[I-10] Redemptions are priced at the live NAV with no front-running protection, unlike deposits, letting a holder exit ahead of a NAV-loss publish	Resolved
[I-11] <code>AccountableYield::_accruePenalties</code> includes <code>accruedPenalties</code> in the penalty base, charging compound interest instead of simple interest	Resolved
[G-1] <code>DepositGateway::_passEpoch</code> re-derives the <code>epochs[epochId]</code> storage pointer the callers already hold	Resolved
[G-2] <code>DepositGateway::_passEpoch</code> re-reads strategy NAV every caller already fetched	Resolved
[G-3] <code>AccountableYield::_accrueFees</code> re-reads the vault <code>totalSupply</code> already loaded by <code>_accruedFeeShares</code>	Resolved

7 Findings

7.1 Medium Risk

7.1.1 `DepositGateway::settle` prices escrowed deposits at the live share price, not the epoch's closing NAV

Description: The gateway batches deposits into epochs so they "enter `AccountableYield` only at the post-NAV price" (`DepositGateway` contract `NatSpec`, `src/modules/DepositGateway.sol:19`). However, `_passEpoch` records only a timestamp into `epoch.navMeasuredAt` (`src/modules/DepositGateway.sol:271,276`) and never stores a share price. `settle` is permissionless, requires only `epoch.status == Passed` (`src/modules/DepositGateway.sol:166`), and mints by calling `IAccountableVault(vault).deposit(...)` (`src/modules/DepositGateway.sol:175`), which routes to `AccountableYield::onDeposit` and prices at the `LIVE _sharePrice(share)` computed at the settle block (`src/strategies/AccountableYield.sol:423,617-623`). `settle` has no deadline and `cancelDeposit` reverts once the epoch is past `Open` (`src/modules/DepositGateway.sol:121`). So the price a depositor realizes is whatever the live price is when their request is settled, not the NAV that closed their epoch.

The protocol mitigates this operationally: a keeper bot settles each passed epoch promptly after the NAV update that closed it, so under normal operation depositors are settled at approximately the closing NAV. The residual is the window between an epoch passing and the keeper settling it. Because NAV updates are not on a fixed schedule and `settle` has no on-chain time bound, if a later `publishRate` lands before the keeper settles a request (keeper lag, downtime, or two NAV updates close together), the still-unsettled depositor can settle into the newer price. `settle` is also paginated for large epochs, so requests in a big batch stay unsettled across several blocks. Continuous management-fee accrual additionally means even a prompt settle lands slightly below the closing price.

Files:

- `DepositGateway::settle`, `_passEpoch` (`src/modules/DepositGateway.sol:163-219,269-277`)
- `AccountableYield::onDeposit`, `_sharePrice` (`src/strategies/AccountableYield.sol:405-427,617-623`)

Impact: When a depositor settles at a price below their epoch's closing NAV, they mint more shares than their deposit warrants, and the excess is diluted away from existing vault shareholders - not from the keeper operator or the protocol. The likelihood is materially reduced by the keeper bot, but the exploit is bounded by keeper uptime and NAV-update spacing rather than by code: a stale passed epoch remains settleable at an arbitrarily later price until someone settles it, so the worst-case window is unbounded and the loss falls on passive LPs who have no involvement in the keeper.

The same defect also lets a keeper discriminate between depositors in a single epoch. `settle` is permissionless, takes a caller-supplied `requestIds` subset, and mints at the live price (`src/modules/DepositGateway.sol:163,168-170`), so the keeper chooses both which requests to settle and in which block. It can settle a favored cohort while the price is low and defer the rest to a higher-priced block, leaving co-epoch depositors who expected one price with different share amounts. The frozen-price mitigation below closes this too: every request in an epoch would settle against the same snapshot.

Recommended Mitigation: Add an on-chain staleness guard so the keeper is the happy path and the contract is the backstop: in `settle`, once `strategy.lastNavMeasuredAt()` has advanced past the epoch's `navMeasuredAt`, reject the settle and route the request to `refund` / re-request instead of minting at the newer live price. When the keeper settles before the next NAV update the guard never triggers and behavior is unchanged; when the keeper misses, the request is refunded rather than settled at an exploitable price. This preserves the simple live-pricing design while bounding the residual to a refund rather than LP dilution. Alternatively, snapshot the share price at `_passEpoch` and mint against that frozen value in `settle`.

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified. `settle` invalidates the epoch (`EpochStatus.Invalid`) if a newer NAV landed after close; those requests must refund, not mint at the new price.

7.1.2 `DepositGateway::cancelDeposit` is a costless NAV-direction option that lets a depositor front-run `AccountableYield::publishRate` to avoid unfavorable pricing

Description: The gateway's stated purpose is to "batch subscriptions into epochs so deposits enter `AccountableYield` only at the post-NAV price" (NatSpec at `src/modules/DepositGateway.sol:19`): a depositor must commit capital before the NAV that prices them is known, so they cannot cherry-pick a favorable price. That commitment is not enforced. An Open-epoch request can be cancelled, for free, at any time while the epoch is Open: `cancelDeposit` requires only `epoch.status == EpochStatus.Open`, returns 100% of the escrow with no fee, penalty, or cooldown, and credits it to the request owner (`src/modules/DepositGateway.sol:113-132`). The epoch only closes when `publishRate` runs on `NavUpdate` (`src/strategies/AccountableYield.sol:239-245`), and `publishRate(newDeployedValue, measuredAt)` is a single, observable transaction whose `newDeployedValue` argument fully determines the resulting share price - it overwrites `deployedAssets` directly (`src/strategies/AccountableYield.sol:218-220`), and the price is `totalAssets / totalSupply`.

A depositor holding an Open-epoch request can therefore read the pending `publishRate` calldata in the mempool, compute the post-NAV share price, and act one-sidedly: leave the request standing when the move is favorable (NAV down implies a lower price implies more shares per asset), or front-run the publish with `cancelDeposit` and recover the full escrow when the move is unfavorable (NAV up implies a higher price implies fewer shares per asset). The cancel-cancel window ends only at the atomic pricing event, so the depositor always exits before being priced against. This is a one-sided directional option on the strategy NAV, written by the protocol to every Open-epoch depositor at zero premium - the precise behavior the batch-at-post-NAV design exists to prevent. The contract should not let a request be cancelled cost-free across the pricing boundary; the absence of that commitment cutoff is the bug, not a flaw in any present mechanism.

Files:

```
DepositGateway::cancelDeposit, requestDeposit          (src/modules/DepositGateway.sol:113-132,
src/modules/DepositGateway.sol:84-110); AccountableYield::publishRate (src/strategies/AccountableYield.sol:21
246)
```

Impact: Existing vault shareholders are diluted. Depositors enter only on NAV declines - minting shares at a cheaper-than-fair price right at a down-NAV print, value that should accrue to existing holders - and abort cost-free on NAV gains. The "post-NAV price" guarantee the gateway exists to provide is defeated and the batching restriction is reduced to cosmetic. The magnitude per cycle is the share-price move the depositor selectively captures, repeatable every NAV cycle; it is bounded by NAV volatility between epoch open and the pricing publish, so the per-event extraction is not parameter-grounded to a fixed figure, but the option is risk-free (the only cost is the cancel gas) and persistent across every cycle.

Proof of Concept:

1. The manager has wired the gateway (`setDepositGateway`) and the loan is `OngoingDynamic`. Eve calls `requestDeposit(100_000e6)`; the request joins the current Open epoch E at price $P_0 = 1.00$. Her escrow is fully refundable via `cancelDeposit` for as long as E is Open. Cost so far: gas only.
2. The `dvnPublisher` broadcasts `publishRate(newDeployedValue, measuredAt)`. The transaction sits in the mempool with its calldata visible.
3. Eve computes the resulting share price from `newDeployedValue` (it overwrites `deployedAssets`, driving `_sharePrice`).
4. If the price would rise to 1.05, the deposit would mint Eve ~95_238 shares instead of the 100_000 at P_0 - unfavorable. Eve submits `cancelDeposit(requestId)` with a higher priority fee, front-running the publish. The epoch is still Open, so the cancel succeeds and all escrow is returned; the publish prices nothing for her.
5. If instead the price would fall to 0.95, Eve does nothing. The epoch passes and her request later settles minting ~105_263 shares - she captured the loss-driven discount on freshly minted shares, value transferred from incumbent holders.

Repeating each NAV cycle, Eve only ever takes the favorable side, at zero premium. The third-party grieving direction (cancelling someone else's request) is blocked by the `req.user != msg.sender` check at `src/modules/DepositGateway.sol:116`, so only the self-benefit direction applies.

Recommended Mitigation: Remove `cancelDeposit` entirely and treat a deposit request as a firm commitment: the depositor is priced at the next NAV and, if that price is unfavorable, exits afterward through the strategy's normal (async) withdrawal path, exactly the trade-off a direct vault deposit already makes.

Because removing cancellation eliminates the only open-epoch exit, pair it with an escape hatch for an open epoch that is never priced (for example a timeout-gated cancel or refund): with cancellation removed and no NAV update, neither `forcePassEpoch` nor `refund` can release open-epoch escrow, so a stalled `publishRate` would otherwise lock a depositor's principal indefinitely. Escrow must continue to route only to the request owner.

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified. Cancel split into `requestCancelDeposit` + `approveCancelDeposit` (onlyManager), gated by `acceptingCancellations` (default false).

7.1.3 AccountableYield lacks an InDefaultClaims wind-down and permanently strands unfulfilled redemptions after a default

Description: This finding is outside the PR75 change set - the default lifecycle, the redemption hooks, and `_sharePrice` are pre-existing `AccountableYield` and `AccountableStrategy` code, not the PR75 diff - and is included because it materially affects depositor funds.

When a loan default is accepted, `loanState` leaves the ongoing states and `publishRate` can no longer run, because it begins with `_requireLoanOngoing` (`src/strategies/AccountableYield.sol:213`). `publishRate` is the only writer of `navGraceDeadline`, so within the grace period (`DEFAULT_NAV_GRACE_PERIOD`, up to 24 hours) `_navIsStale` becomes permanently true and can never be cleared. From that point a redemption that has not already been fulfilled can never be fulfilled, because all three fulfillment routes are dead:

1. Instant fulfillment in `requestRedeem` requires `!_navIsStale()` in `onRequestRedeem` (`src/strategies/AccountableYield.sol:473`), which is false, so the request only queues.
2. Queue processing through `vault.processUpToShares` is reachable only via `_processAvailableWithdrawals`, called solely by `repay` and `accrueAndProcess` (`src/strategies/AccountableYield.sol:365`, `src/strategies/AccountableYield.sol:400`), both of which carry `whenNotStale` and `_requireLoanOngoing` and therefore revert.
3. `AccountableYield` does not implement the `InDefaultClaims` wind-down its sibling strategies provide. `AccountableFixedTerm` and `AccountableOpenTerm` reprice `_sharePrice` to `IAccountableVault::assetShareRatio` and allow instant non-NAV-gated redemption in the `Repaid` and `InDefaultClaims` states, but `AccountableYield` overrides neither path and never reads `InDefaultClaims` at all.

`InDefaultClaims` is terminal - there is no transition back to an ongoing state - so the staleness and the resulting freeze are permanent absent a contract upgrade.

Files:

- `AccountableYield::publishRate`, `onRequestRedeem`, `coverDefault` (`src/strategies/AccountableYield.sol`)
- `AccountableYield::_processAvailableWithdrawals`, `_sharePrice` (`src/strategies/AccountableYield.sol`)
- `AccountableAsyncRedeemVault::requestRedeem`, `redeem`, `withdraw` (`src/vault/AccountableAsyncRedeemVault.sol`)

Impact: Depositors whose redemptions are not fulfilled before the NAV goes stale are permanently unable to redeem their shares for assets. The safety-module coverage injected through `coverDefault` - the capital specifically provided to backstop holders during a default - is unreachable for that cohort, because reaching it requires fulfillment (reserving liquidity against the queue), which is the blocked step. `cancelRedeemRequest` only returns the escrowed shares, not assets, so it is not an exit. The loss is bounded: redemptions already fulfilled remain claimable at any time, and during the brief post-default window before `navGraceDeadline` lapses holders can still exit via instant fulfillment against the remaining reserves and the coverage, though at the un-written-down `_sharePrice` and first-come-first-served, which lets early exiters over-extract. So the impact is permanent loss of access for late or illiquid redemptions during exactly the scenario the coverage mechanism exists to handle, rather than outright theft. Triggering it requires a default to be initiated (a privileged `defaultLoan` plus the permissionless `acceptDefault` after the one-day timer), but the presence of `coverDefault` shows defaults are a designed-for path.

Recommended Mitigation: Implement the `InDefaultClaims` wind-down the sibling strategies already have: in `_sharePrice` return `IAccountableVault(vault).assetShareRatio()` when `loanState` is `Repaid` or `InDefaultClaims`, and allow instant non-NAV-gated redemption in those states (mirroring `_isInstantRedeem` and `_requireCanRequestRedeem`). This lets depositors claim their pro-rata share of the remaining real assets, including the `coverDefault` coverage, at the recoverable basis regardless of NAV staleness. Alternatively, if `AccountableYield` is never intended to enter default, disable the inherited default entrypoints (`defaultLoan`, `coverDefault`, and the base `acceptDefault`) the same way `prepay` and `pay` are disabled with `revert NotSupportedByStrategy()`, so the unhandled terminal state is unreachable.

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified. Instant redeem allowed when `loanState` is in `InDefaultClaims`.

7.1.4 AccountableStrategy::acceptDefault has no loan-state guard, letting a fully repaid loan be forced into InDefault and bricking the vault

Description: `acceptDefault` contains only a timelock check and no guard on the current `loanState` (and is external with no access modifier, so any caller can invoke it):

```
function acceptDefault() external {
    if (block.timestamp < _defaultValidAt || _defaultValidAt == 0) revert TimelockNotExpired();
    loanState = LoanState.InDefault;
    emit LoanDefaulted(_loan.outstandingPrincipal, IAccountableVault(vault).totalAssets());
}
```

`defaultLoan` arms the timelock and leaves `loanState` unchanged:

```
function defaultLoan() external onlySafetyModuleOrManager {
    _requireLoanOngoing();
    if (_defaultValidAt != 0) revert DefaultAlreadyPending();
    _defaultValidAt = block.timestamp + 1 days;
    emit LoanDefaultInitiated();
}
```

If the borrower then repays in full before the timelock expires, `repay` transitions the loan to `Repaid` but never clears `_defaultValidAt`:

```
if (_loan.outstandingPrincipal == 0 && accruedPenalties == 0 && deployedAssets == 0) {
    loanState = LoanState.Repaid;
}
```

The only function that clears `_defaultValidAt` is `rejectDefault`:

```
function rejectDefault() external onlyManager {
    _defaultValidAt = 0;
    emit LoanDefaultRejected();
}
```

So once the 1-day timelock elapses, any caller can invoke `acceptDefault`, which unconditionally overwrites `loanState` from `Repaid` to `InDefault`. This corrupts the loan state machine: from `InDefault` the only forward move is `coverDefault`, which sinks the loan into `InDefaultClaims`, and there is no transition back to an ongoing state. `borrow`, `repay`, `publishRate`, `deposits`, and `queue processing` all gate on `_requireLoanOngoing()` and `revert` permanently.

Impact: Any unprivileged caller can permanently brick the vault by forcing a fully repaid loan into a terminal default state (where `borrow`, `repay`, `deposits`, `NAV publishing`, and `queue processing` all `revert` with no recovery path) whenever a default was initiated and the manager did not call `rejectDefault` before the borrower's pre-timelock repayment, a sequence an attacker can engineer by timing the repayment just before the deadline.

Proof of Concept: `alice` deposits 100,000 USDC, the borrower draws it all, the manager initiates a default, the borrower repays in full (driving `loanState` to `Repaid`), and after the timelock an arbitrary attacker calls `acceptDefault`

and forces InDefault.

Add the following test to test/strategies/PoC_RepaidForcedDefault.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./AccountableStrategyBase.t.sol";

contract PoC_RepaidForcedDefault is AccountableStrategyBaseTest {
    function _setupYieldLoan() internal {
        vm.prank(manager);
        usdcYieldLoan.setPendingBorrower(borrower);
        vm.prank(borrower);
        usdcYieldLoan.acceptBorrowerRole();
        vm.prank(manager);
        usdcYieldLoan.setOperationsAdminEnabled(true);
        address operationsAdmin = globals.operationsAdmin();
        vm.prank(operationsAdmin);
        usdcYieldLoan.setDVNPublisher(navOracle);

        LoanTerms memory terms = LoanTerms({
            minDeposit: 0, minRedeem: 0, maxCapacity: USDC_AMOUNT * 10, minCapacity: 0,
            interestRate: 0, interestInterval: 30 days, duration: 0, depositPeriod: 0,
            acceptGracePeriod: 0, lateInterestGracePeriod: 2 days, lateInterestPenalty: 5e2,
            withdrawalPeriod: 0
        });
        vm.prank(manager);
        usdcYieldLoan.setTerms(terms);
        vm.prank(borrower);
        usdcYieldLoan.acceptTerms();
    }

    function test_RepaidLoanCanBeForcedIntoDefault() public {
        vm.warp(1739893670);
        _setupYieldLoan();

        uint256 depositAmount = 100_000 * 10 ** USDC_DECIMALS;
        vm.prank(alice);
        usdcYieldVault.deposit(depositAmount, alice, alice);
        vm.prank(borrower);
        usdcYieldLoan.borrow(depositAmount);
        assertEq(uint8(usdcYieldLoan.loanState()), uint8(LoanState.OngoingDynamic));

        // Manager initiates default: only arms _defaultValidAt = now + 1 days.
        vm.prank(manager);
        usdcYieldLoan.defaultLoan();

        // Timelock genuinely pending.
        vm.expectRevert(TimelockNotExpired.selector);
        usdcYieldLoan.acceptDefault();

        // Borrower repays IN FULL before the timelock -> Repaid, but _defaultValidAt is NOT cleared.
        vm.prank(borrower);
        usdcYieldLoan.repay(depositAmount);
        assertEq(uint8(usdcYieldLoan.loanState()), uint8(LoanState.Repaid));
        assertEq(usdcYieldLoan.loan().outstandingPrincipal, 0);
        assertEq(usdcYieldLoan.accruedPenalties(), 0);
        assertEq(usdcYieldLoan.deployedAssets(), 0);

        // Manager forgot rejectDefault(); timelock elapses.
        vm.warp(block.timestamp + 1 days + 1);
    }
}
```

```

    // BUG: an arbitrary caller forces Repaid -> InDefault.
    vm.prank(address(0xBADBADBAD));
    usdcYieldLoan.acceptDefault();
    assertEq(uint8(usdcYieldLoan.loanState()), uint8(LoanState.InDefault));

    // Vault is bricked: ongoing-gated operations revert.
    vm.prank(borrower);
    vm.expectRevert(LoanNotOngoing.selector);
    usdcYieldLoan.borrow(1);

    // coverDefault sinks it into InDefaultClaims with no recovery path.
    vm.prank(manager);
    usdcYieldLoan.coverDefault(0);
    assertEq(uint8(usdcYieldLoan.loanState()), uint8(LoanState.InDefaultClaims));
  }
}

```

Run with: `forge test --match-test test_RepaidLoanCanBeForcedIntoDefault -vv`

Output:

```

Ran 1 test for test/strategies/PoC_RepaidForcedDefault.t.sol:PoC_RepaidForcedDefault
[PASS] test_RepaidLoanCanBeForcedIntoDefault() (gas: 786153)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.37s

```

Recommended Mitigation: Guard `acceptDefault` on the loan state, and clear the pending timer when the loan reaches `Repaid`:

```

function acceptDefault() external {
    if (block.timestamp < _defaultValidAt || _defaultValidAt == 0) revert TimelockNotExpired();
+   _requireLoanOngoing(); // a repaid loan is no longer default-eligible
    loanState = LoanState.InDefault;
    ...
}

```

```

if (_loan.outstandingPrincipal == 0 && accruedPenalties == 0 && deployedAssets == 0) {
    loanState = LoanState.Repaid;
+   _defaultValidAt = 0; // cancel any pending default so it cannot be accepted later
}

```

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified. `acceptDefault` now reverts `LoanNotOngoing` for a non-ongoing loan, and `_defaultValidAt` is cleared on the `Repaid` transition so a repaid loan can no longer be forced into `InDefault`.

7.2 Low Risk

7.2.1 `DepositGateway::settle, refund` leave escrow stuck when the strategy is stale or paused and `_refundable` recognizes neither, permanently inflating `unsettledEscrow`

Description: When the strategy is NAV-stale or paused, the deposit settlement path has no exit for an otherwise-deliverable request. `DepositGateway::settle` (`src/modules/DepositGateway.sol:163-192`) calls `IAccountableVault::deposit`, which routes through `AccountableYield::onDeposit` (`src/strategies/AccountableYield.sol:405-412`). That hook carries both `whenNotStale` and `whenNotPaused`, so it reverts while the NAV grace period has expired or the strategy is paused. The settle loop swallows that revert in its `try/catch` and simply leaves the request pending. The natural escape - refund - does not help: `_refundable` (`src/modules/DepositGateway.sol:280-302`) enumerates loan-state, gateway-mismatch, capacity, KYC, Whitelist, and lowered-minDeposit branches, but treats neither stale NAV nor pause as refundable, so refund reverts `NotRefundable`. The request is therefore neither settleable nor refundable for the duration of the stale/paused window.

Files:

- `DepositGateway::settle, refund` (`src/modules/DepositGateway.sol`)
- `DepositGateway::_refundable` (`src/modules/DepositGateway.sol`)
- `AccountableYield::onDeposit` (`src/strategies/AccountableYield.sol`)

Impact: A whitelisted, correctly-sized request that should settle cannot settle and cannot be refunded while the strategy is stale or paused. Because the request stays pending, its escrow remains summed into `unsettledEscrow`, which `requestDeposit` uses as the capacity reservation (`assets + unsettledEscrow > maxAssets`). The stuck escrow throttles available headroom for every other depositor until a new NAV is published or the strategy is unpaused. The condition is transient and self-resolves once the strategy returns to a fresh, unpaused state, so funds are recoverable rather than lost.

Recommended Mitigation: Add a refundable branch for the stale/paused condition so a depositor can recover escrow when settlement is blocked by strategy state rather than by their own eligibility. Either expose the strategy's stale/paused state to `_refundable` and return `true` when settlement is blocked by it, or allow the depositor to cancel a Passed-epoch request back to themselves under that condition. Preserve the existing branches so eligibility-based refunds continue to work.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: This leaves a minor grieving vector open where anyone could refund someone while the vault is paused, even if the user would want to wait and join once it's unpaused. They'd then have to wait yet another nav update

Accountable: Fixed in commit [2b60b00](#)

Cyfrin: Verified.

7.2.2 `AccountableYield::updateLateStatus` is permissionless and lacks the `whenNotStale` and `whenNotPaused` guards

Description: `AccountableYield::updateLateStatus` (`src/strategies/AccountableYield.sol:497-500`) is permissionless and carries no `whenNotStale` or `whenNotPaused` guard. It calls `_updateDelinquentStatus`, which when the loan is already delinquent calls `_accruePenalties` (`src/strategies/AccountableYield.sol:712-735`), a wall-clock accumulator that charges penalty for every second elapsed since the last penalty timestamp. The borrower's only cure path, `repay` (`src/strategies/AccountableYield.sol:327`), is gated `whenNotStale` `whenNotPaused`. While the strategy is paused or its NAV is stale, the borrower cannot repay, but any third party can keep calling `updateLateStatus` to keep advancing the penalty accumulator over the same window.

Files:

- `AccountableYield::updateLateStatus` (`src/strategies/AccountableYield.sol`)
- `AccountableYield::_accruePenalties, repay` (`src/strategies/AccountableYield.sol`)

Impact: During a pause or stale-NAV window the borrower is locked out of repayment while penalties continue to compound against the position. The penalties accrued over that window are charged to a position the borrower had no opportunity to cure, and `accruedPenalties` is only reduced inside `repay`, so the borrower must absorb penalties that accrued precisely when repayment was impossible. The accrual is bounded by the duration of the stale/paused window and the configured penalty rate; the borrower cannot retroactively reverse penalties already booked over that window.

Recommended Mitigation: Gate the penalty-advancing path so the delinquency clock does not run while the borrower's cure path is blocked. Apply `whenNotStale` and `whenNotPaused` to `updateLateStatus` (consistent with `repay`), or suspend penalty accrual in `_accruePenalties` while the strategy is paused or NAV is stale, resuming the clock from the moment repayment becomes possible again.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.2.3 `DepositGateway::forcePassEpoch` is permissionless and can strip a pending depositor's cancel window

Description: `DepositGateway::forcePassEpoch` (`src/modules/DepositGateway.sol:147-160`) is permissionless and succeeds as soon as `strategy.lastNavMeasuredAt()` exceeds the epoch's `openNavMeasuredAt`. A pending deposit can be cancelled only while its epoch is `Open`: `cancelDeposit` (`src/modules/DepositGateway.sol:113-132`) reverts `EpochNotOpen` once the epoch leaves `EpochStatus.Open`.

In the normal flow a NAV publish closes the epoch atomically: `publishRate` calls `onNavUpdate`, which passes the epoch in the same transaction, so the cancel window closes with the publish itself and no `Open` epoch remains for `forcePassEpoch` to act on. The distinct case `forcePassEpoch` targets is a missed notification: `publishRate` wraps `onNavUpdate` in a `try/catch` (`GatewayNavUpdateFailed`), so if that notify reverts, `lastNavMeasuredAt` advances while the epoch stays `Open`. In that window a third party can call `forcePassEpoch` to move the epoch from `Open` to `Passed`, locking in every pending request at the already-published NAV before its depositor has a chance to cancel.

Files:

- `DepositGateway::forcePassEpoch` (`src/modules/DepositGateway.sol`)
- `DepositGateway::cancelDeposit` (`src/modules/DepositGateway.sol`)

Impact: A depositor who would have cancelled after an adverse NAV publish can be front-run out of their cancel window by anyone calling `forcePassEpoch` in the same block or shortly after. The request then settles at the post-publish NAV rather than being refundable on demand. The depositor still receives shares at the published price, so this is a loss of optionality (the ability to back out at the last moment) rather than a loss of principal; the user does not gain the right to cancel back after `Passed`.

Recommended Mitigation: Treat a deposit request as a firm commitment and remove the cancellation path entirely, so there is no last-moment cancel window for a force-pass to strip. The depositor is then priced at the next NAV regardless of who triggers the pass or when, exactly the trade-off a direct vault deposit already makes, and exits afterward through the strategy's normal (async) withdrawal path if the resulting price is unfavorable. With no cancel path, the timing and permissionlessness of `forcePassEpoch` no longer affect the depositor's outcome.

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified.

7.2.4 `DepositGateway::requestDeposit` `minDeposit` gate is weaker than the vault's `MIN_AMOUNT_WEI` floor, leaving sub-floor requests permanently unsettlable and non-refundable

Description: `DepositGateway::requestDeposit` (`src/modules/DepositGateway.sol:84-110`) enforces `assets >= loan.minDeposit` but does not enforce the vault's own minimum-amount floor (`MIN_AMOUNT_WEI`, 10_000) that `IAccountableVault::deposit` checks at settle time. If the manager sets `loan.minDeposit` below that floor, a request escrowed at an amount between the two (e.g. `minDeposit <= assets < MIN_AMOUNT_WEI`) is accepted, but at settle the vault's minimum-amount check reverts and the `try/catch`

in `settle` (`src/modules/DepositGateway.sol:163-192`) swallows it. The request is then not refundable either: `_refundable`'s `lowered-minDeposit` branch (`src/modules/DepositGateway.sol:280-302`) tests `strategy_.loan().minDeposit > assets`, which is false for a request at or above `minDeposit`, and no other branch matches a sub-floor amount.

The same floor gap also admits zero-asset requests at its lower extreme. With `minDeposit` configured as zero, `assets == 0` satisfies `assets >= minDeposit`, and because `safeTransferFrom` moves no tokens the request costs only gas. A submitter can then create an unbounded number of zero-asset requests, each likewise unseizable (the vault rejects the zero amount) and unrefundable, each pinning `pendingCount` above zero so the epoch can never finalize. The zero-asset variant escrows nothing, so its harm is settlement grieving and keeper-gas inflation rather than stranded funds, but it stems from the same missing floor check.

Files:

- `DepositGateway::requestDeposit` (`src/modules/DepositGateway.sol`)
- `DepositGateway::_refundable` (`src/modules/DepositGateway.sol`)

Impact: A request sized between the gateway's `minDeposit` and the vault's hard minimum can be permanently stuck: it cannot settle (vault rejects sub-floor amounts) and cannot be refunded (`_refundable` returns false for it). The escrow stays summed into `unsettledEscrow`, and because the request keeps `pendingCount` above zero, its epoch never finalizes. The depositor cannot recover the escrowed amount through the contract's own paths; recovery would require a manual intervention outside the defined flow. Triggering it requires the manager to have configured `minDeposit` below the vault floor.

Recommended Mitigation: Enforce the vault's minimum-amount floor at request time so the gateway never accepts an amount the vault will later reject - either validate `assets >= MIN_AMOUNT_WEI` in `requestDeposit` alongside the `minDeposit` check, or constrain `setTerms/updateTerms` so `minDeposit` can never be set below the vault floor. Additionally, add a `_refundable` branch covering amounts the vault would reject so any stranded request retains a refund path.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.2.5 `AccountableYield::_accruedFeeShares` degenerate fee-overflow early return advances the high-water mark without charging the fee

Description: `AccountableYield::_accruedFeeShares` (`src/strategies/AccountableYield.sol:558-603`) returns early with zero fee shares when `totalFeeAssets >= newTotalAssets`, but still returns the full `newTotalAssets`. Back in `_accrueFees` (`src/strategies/AccountableYield.sol:503-516`), the high-water-mark block then runs unconditionally: with `performanceFeeShares` and `managementFeeShares` both zero, `hwmSupply` equals the current `totalSupply`, so `postFeePrice = newTotalAssets.mulDiv(PRECISION, hwmSupply)` is the gross, un-fed share price. If that price exceeds `peakSharePrice`, the high-water mark is ratcheted up to the gross price even though no performance fee was charged on the underlying gain.

Files:

- `AccountableYield::_accruedFeeShares`, `_accrueFees` (`src/strategies/AccountableYield.sol`)

Impact: In the degenerate case where computed fees meet or exceed total assets, a genuine gain advances `peakSharePrice` to the gross post-gain price without that gain ever being taxed. Because the performance fee only applies to gains above the high-water mark, the un-taxed gain is permanently forgiven - a later, smaller gain measured against the inflated mark yields a reduced or zero performance fee. The protocol loses fee revenue it was otherwise entitled to; depositors are not harmed. The condition requires fees to compute to at least total assets, which is an extreme regime.

Recommended Mitigation: Advance the high-water mark only when fees were actually charged. Skip the `peakSharePrice` update when `_accruedFeeShares` took the degenerate early return (e.g. have it signal that no fee was charged), so the high-water mark is not ratcheted past a gain that escaped the performance fee. Ensure the normal path, where fees are minted, continues to advance the mark using the post-fee price.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.2.6 DepositGateway::requestDeposit capacity reservation lets one requester exhaust headroom and block all other depositors

Description: DepositGateway::requestDeposit (src/modules/DepositGateway.sol:84-110) reverts ExceedsMaxDeposit when $\text{assets} + \text{unsettledEscrow} > \text{maxAssets}$, reserving capacity against the running unsettledEscrow. An actor can size a single request to consume all remaining headroom, after which every other requestDeposit reverts for lack of capacity. While the epoch is still Open, the actor then calls cancelDeposit (src/modules/DepositGateway.sol:113-132), which returns their full escrow and decrements unsettledEscrow. The sequence is self-reversing: the griever recovers their funds while having blocked others for the interval.

Files:

- DepositGateway::requestDeposit, cancelDeposit (src/modules/DepositGateway.sol)

Impact: A single actor can repeatedly deny other users the ability to subscribe to an epoch at near-zero net cost, since the blocking escrow is fully recoverable via cancel. This is a repeatable, self-reversing denial of service against other subscriptions, gated by the available capacity headroom. No funds are lost - the griever's escrow returns to them and victims keep their tokens - but legitimate depositors are denied entry while the headroom is held; the condition is fully recoverable once the griever cancels or the epoch advances.

Recommended Mitigation: Remove the cost-free cancellation that makes the reserve-then-recover loop repeatable, and treat a deposit request as a firm commitment that settles at the next NAV rather than escrow the requester can reclaim on demand. Without a free cancel, reserving the remaining headroom requires committing capital that is actually deposited and minted into shares at settlement, so an actor can no longer hold and release shared capacity at will to deny other subscriptions: blocking an epoch would cost a real, non-recoverable deposit per epoch instead of being free and repeatable. If preventing a single committed deposit from occupying an epoch's entire headroom is also a concern, additionally impose a per-account capacity limit within an epoch so no single request can monopolize the shared headroom.

Accountable: Fixed in commit [22bf5d5](#)

Cyfrin: Verified.

7.2.7 AccountableYield::acceptTerms writes new penalty parameters before accruing, mispricing the proposal-to-acceptance window

Description: The manager-side updateTerms correctly calls _accruePenalties() before storing the pending terms (line 123), settling the period up to the proposal at the old rate. The borrower-side acceptTerms does the opposite. In the ongoing-loan branch it writes the new lateInterestPenalty and lateInterestGracePeriod into _loan first (lines 161-162) and only then calls _updateDelinquentStatus (line 166), which fires _accruePenalties using the already-mutated values:

```
_loan.lateInterestPenalty = terms.lateInterestPenalty;
_loan.lateInterestGracePeriod = terms.lateInterestGracePeriod; // new (extended) grace
...
_updateDelinquentStatus(); // -> _accruePenalties() reads the NEW grace period
```

_accruePenalties computes $\text{graceEnd} = \text{delinquencyStart} + \text{_loan.lateInterestGracePeriod}$. With the extended grace period, if $\text{block.timestamp} \leq \text{graceEnd}$ it returns early, accruing zero penalties for the whole window since the last accrual. Even a partial extension skips the segment between _lastPenaltyTime and the new graceEnd, because $\text{penaltyStart} = \max(\text{_lastPenaltyTime}, \text{graceEnd})$.

The same ordering also misprices in the opposite direction. If the accepted terms raise lateInterestPenalty and the window is already past grace ($\text{block.timestamp} > \text{graceEnd}$), _accruePenalties reads the new lateInterestPenalty (written at line 161 before the accrual fires) and charges the entire proposal-to-acceptance window at the new higher rate ($\text{penaltyAmount} = \text{_totalAssets} * \text{newRate} * \text{penaltyTime}$) rather than the rate that was actually in effect during that window.

Impact: Whichever way the terms move, the proposal-to-acceptance window is settled under the new parameters rather than the old ones: extending the grace period erases penalty income owed to depositors for that window, while raising the rate overcharges the borrower for it.

Recommended Mitigation: Settle penalties at the old terms before mutating them, mirroring the ordering already used in `updateTerms`:

```
+ // Settle penalties at the OLD terms before overwriting them (mirror updateTerms)
+ _accruePenalties();
+
    _loan.minRedeem = terms.minRedeem;
    _loan.minDeposit = terms.minDeposit;
    _loan.maxCapacity = terms.maxCapacity;
    _loan.interestInterval = terms.interestInterval;
    _loan.lateInterestPenalty = terms.lateInterestPenalty;
    _loan.lateInterestGracePeriod = terms.lateInterestGracePeriod;
```

Accountable: Fixed in commit [c54cb88](#)

Cyfrin: Verified. `acceptTerms` now accrues penalties against the existing parameters before writing the new ones, so the proposal-to-acceptance window is priced correctly.

7.3 Informational

7.3.1 `DepositGateway::requestDeposit` omits the constructor's KYC rejection on a later Whitelist-to-KYC flip

Description: The `DepositGateway` constructor (`src/modules/DepositGateway.sol:64-81`) reverts `SigAuthNotSupported` if the vault's `permissionLevel` is `KYC` at deploy time, because the gateway cannot supply the per-account EIP-712 signature a `KYC` vault requires. But `permissionLevel` is mutable on the vault. After a flip from `Whitelist` (or other) to `KYC`, `requestDeposit` (`src/modules/DepositGateway.sol:84-110`) keeps accepting escrow: its `_requireWhitelistIfConfigured` gate (`src/modules/DepositGateway.sol:319-335`) only enforces anything when the level is `Whitelist` and is a no-op for `KYC`. Settlement, however, is impossible - `IAccountableVault::deposit` under `KYC` needs a signature the gateway never carries. Open-epoch requests escrowed after the flip remain cancellable, but once their epoch passes they can never settle.

Files:

- `DepositGateway::requestDeposit` (`src/modules/DepositGateway.sol`)
- `DepositGateway::_requireWhitelistIfConfigured` (`src/modules/DepositGateway.sol`)

Impact: After a vault permission flip to `KYC`, the gateway continues to accept deposits it cannot ever settle. A request escrowed post-flip that reaches a `Passed` epoch is stuck against settlement; `_refundable` (`src/modules/DepositGateway.sol:280-302`) does return `true` on its `KYC` branch, so refund remains available as a recovery path, but the deposit flow itself is broken for that gateway from the moment of the flip. The trigger is an admin-driven permission change rather than an attacker; the impact is a silently broken deposit path that keeps accepting funds.

Recommended Mitigation: Mirror the constructor's `KYC` rejection at request time: have `requestDeposit` revert when the vault's current `permissionLevel` is `KYC`, so the gateway stops accepting escrow it cannot settle the instant the vault flips. This surfaces the misconfiguration to depositors immediately instead of accepting deposits into a dead-end settlement path.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.2 `DepositGatewayFactory::createDepositGateway` is permissionless and registry-less, creating a poisoned-gateway social-engineering surface

Description: `createDepositGateway(strategy_)` deploys a new `DepositGateway` bound to the caller-supplied `strategy_` and emits `DepositGatewayCreated(gateway, strategy_)` with no validation of `strategy_` at the factory layer and no registry of legitimate gateways. The deployed `DepositGateway` constructor backstops the obvious cases (it reverts on a zero or non-conforming strategy and rejects `KYC` vaults), and a gateway can only be activated when `AccountableYield::setDepositGateway` confirms `gateway.strategy() == address(this)` under `onlyManager`. So a gateway built around a garbage strategy can never be wired into the protocol path, and there is no direct on-chain fund-loss vector here. The residual concern is observational: because the factory is permissionless and emits the same `DepositGatewayCreated` event for any caller, an attacker can deploy a gateway bound to the *real* strategy address - which will pass the later `gateway.strategy() == this` check - and any off-chain tooling or operator that selects a gateway to wire by scanning `DepositGatewayCreated` events, rather than by their own deployment record, could be steered onto an attacker-deployed gateway. The on-chain wiring step is manager-gated and validated, so this is a social-engineering / operational-hygiene surface, not a contract bug.

Recommended Mitigation: Treat `createDepositGateway` output as untrusted: operators must wire only gateways they deployed and whose address they recorded out-of-band, never a gateway discovered by scanning `DepositGatewayCreated`. If on-chain provenance is desired, have the factory maintain a `mapping(address strategy => address[] gateways)` registry and/or restrict `createDepositGateway` to a known deployer role, so the event stream is not the trust anchor.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.3 `AccountableYield::onMint` entry path is permanently dead once a deposit gateway is wired because the gateway only calls `vault.deposit`

Description: PR75 adds `_requireFromGateway(controller)` to both `AccountableYield::onDeposit`, `onMint`, so once a deposit gateway is wired, every deposit/mint hook must originate from the wired gateway (a direct user is rejected with `DepositNotFromGateway`). `DepositGateway::settle` is the only path that drives escrowed requests into the vault, and it calls `IAccountableVault(vault).deposit/assets, req.user, address(this))` exclusively - it never calls a mint path. The consequence is that with a gateway set, `onMint` is unreachable for all actors: direct users are rejected by `_requireFromGateway`, and the gateway never invokes the mint route. The exact-shares (mint-by-shares) subscription entry that `onMint` implements therefore has no live caller and no replacement, so that entry mode is effectively removed for the gated configuration. This is a structural dead-path / capability-loss observation, not an exploit: no funds are at risk, but a reader expecting `onMint` to remain a usable entry will be surprised, and a future change that begins routing settlement through a mint path would re-activate code that currently receives no test coverage in the gated flow.

Recommended Mitigation: Decide whether mint-by-shares entry is intended to survive the gateway wiring. If it is not, document on `onMint` that the gated configuration disables it and consider removing or explicitly reverting the unused path to avoid future re-activation of untested code. If it is intended, give the gateway a settle variant that routes through the mint hook (mint-by-shares) so the exact-shares entry remains reachable, and cover it with tests.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.4 `AccountableYield` still writes `lostAssets` and `lastTotalAssets` but no on-chain logic reads either after PR75 rewrote the fee path to a per-share HWM

Description: The performance-fee computation in `_accruedFeeShares` derives the fee purely from a per-share high-water mark (`peakSharePrice`), with no reference to either `lostAssets` or `lastTotalAssets` - the two accumulators the pre-rewrite fee model relied on (an absolute total-assets baseline net of realized losses). A scope-wide trace finds both are still written but read nowhere on-chain: `publishRate` increments `lostAssets` on every reported loss (`src/strategies/AccountableYield.sol:230`), and `lastTotalAssets` is rewritten on every flow and accrual - borrow (`src/strategies/AccountableYield.sol:316`), repay (`src/strategies/AccountableYield.sol:363`), `onDeposit` (`src/strategies/AccountableYield.sol:426`), `onMint` (`src/strategies/AccountableYield.sol:452`), and `_accrueFees` (`src/strategies/AccountableYield.sol:507`) - yet the only non-write references to either are the auto-generated public view getters. Both are dead state. The `lostAssets` NatSpec ("Realized losses that should not generate fees on recovery") describes a fee-recovery-suppression role the current path implements through `peakSharePrice` instead, and the `lastTotalAssets += assets` form on the deposit and mint hooks pays an SLOAD plus SSTORE on every deposit for a value nothing consumes. There is no incorrect on-chain result today, since the per-share HWM provides recovery suppression up to the prior peak by construction. The risk is drift plus wasted gas: a maintainer who trusts the stale `lostAssets` comment may re-wire it back into the fee math alongside the HWM, double-counting the recovery suppression and under-charging fees, while every flow path keeps paying to maintain a `lastTotalAssets` baseline the rewrite abandoned.

Recommended Mitigation: Either remove both now-unused accumulators along with their writes (`lostAssets` in `publishRate`; `lastTotalAssets` in `borrow`, `repay`, `onDeposit`, `onMint`, `_accrueFees`), or document that under the per-share high-water-mark model both are retained only as observability figures and are not inputs to fee math. If `lostAssets` is kept, add a code comment at the `peakSharePrice`-based fee computation making explicit that recovery suppression is the HWM's responsibility and that `lostAssets` must not be re-introduced into the fee path. Removing storage from an upgradeable contract changes the slot layout, so do so only on a fresh deployment or with a deliberate slot-preserving migration.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.5 DepositGateway::_refundable carries an inverted minDeposit comment that misstates the setter-regression direction

Description: In `DepositGateway::_refundable`, the final branch is `if (strategy_.loan().minDeposit > assets) return true;`, preceded by the comment "If minDeposit was lowered after the request was made, it is refundable". The comment describes the wrong direction: `loan().minDeposit > assets` is true precisely when `minDeposit` has been *raised* above the amount the user already escrowed, which is the case that strands the request (a settle would now fail the strategy's minimum-deposit check, so the request must be refundable). The code is correct; the comment inverts the condition it documents. The hazard is drift: a future editor "fixing" the code to match the comment - changing the test to `minDeposit < assets` - would make legitimate requests that sit below the *current* `minDeposit` refundable and would stop refunding the raised-minimum requests that actually need it, breaking settlement for the exact population the branch exists to protect.

Recommended Mitigation: Correct the comment to match the code, e.g. "If minDeposit was raised above the escrowed amount after the request was made, the request can no longer be settled and is refundable". Leave the condition `strategy_.loan().minDeposit > assets` unchanged.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.6 Missing or incomplete events: absent emissions, missing indexed params and missing event fields

Description: PR75's event surface has four observability gaps. None is read by on-chain logic and none affects fund safety, but each breaks off-chain reconstruction by indexers, keeper dashboards, or fee-monitoring. The individual gaps:

1. `DepositGateway::settle, refund` emit `EpochSettled` with a stale aggregate `totalAssets`. `epoch.totalAssets` is only ever increased in `requestDeposit` and decreased in `cancelDeposit`; neither `settle` nor `refund` decrements it. So when an epoch is finalized after a mix of settlements and refunds, `EpochSettled(epochId, epoch.totalAssets)` reports the gross at-pass subscribed total, including the portion that was refunded rather than minted into vault shares. Off-chain TVL trackers and analytics that sum `EpochSettled::totalAssets` over-count assets that actually entered the vault by the total of all refunded requests in each epoch, and the discrepancy grows exactly with the operationally-expected refund triggers (de-whitelisting, capacity exceeded, raised minimum deposit).

Source: `src/modules/DepositGateway.sol:190` (emit), `src/modules/DepositGateway.sol:175-184, 207-219` (the settle/refund finalize paths that omit the decrement).

Recommended: track a running settled-assets accumulator on the Epoch and emit that in `EpochSettled` so the event is self-consistent with its name; alternatively rename the field to make explicit it is the gross subscribed total fixed at pass time so indexers do not treat it as assets-deposited-into-vault.

2. `DepositGateway::refund` finalizes an epoch to `Settled` without emitting `EpochSettled`. When the last pending request of a passed epoch is refunded rather than settled, `refund` sets `epoch.status = EpochStatus.Settled` and `epoch.settledAt` once `pendingCount` reaches 0, but emits only the per-request `DepositRefunded` - no `EpochSettled`. The `IDepositGateway NatSpec` documents `EpochSettled` as emitted when every request in an epoch is settled or refunded, so the refund-finalized case is an in-scope emit condition that is missing. Any off-chain epoch-lifecycle state machine that tracks finalization by listening for `EpochSettled` permanently misses the transition and shows the epoch stuck in `Passed`, even though on-chain state reads `Settled`.

Source: `src/modules/DepositGateway.sol:212-215`.

Recommended: inside the `if (epoch.pendingCount == 0)` block in `refund`, emit `EpochSettled(req.epochId, epoch.totalAssets)` consistent with the settle path, or factor the finalize-and-emit into a shared internal helper called from both `settle` and `refund`.

3. `AccountableYield::reinitialize` overwrites the fee high-water mark `peakSharePrice` with no event. `reinitialize` (the one-shot, manager/security-admin-gated migration hook) recomputes and overwrites `peakSharePrice`, which directly governs future performance-fee charges - a material accounting change that can re-enable or suppress fees on the next accrual. No event is emitted, while comparable setters

(setNavGracePeriod, setDepositGateway) all emit dedicated events. A post-migration fee anomaly therefore cannot be correlated to the HWM reset off-chain.

Source: src/strategies/AccountableYield.sol:84-88.

Recommended: emit an event on reinitialization capturing the old and new peakSharePrice so fee-monitoring has a signal the high-water mark was reset.

4. DepositGateway::settle emits RequestSettled with user left un-indexed, while the sibling lifecycle events DepositRequested, DepositCancelled, DepositRefunded all index user. A frontend or indexer wanting all settlements for a given address cannot do a topic filter on the terminal settlement event and must scan and decode every event, an asymmetry inconsistent with the rest of the event family.

Source: src/modules/DepositGateway.sol:180 (emit; event declared in IDepositGateway).

Recommended: mark user as indexed in the RequestSettled event declaration.

Recommended Mitigation: Apply the per-component fixes above: track and emit the actually-settled asset total (or rename the EpochSettled field), emit EpochSettled on the refund-finalized branch, emit an old/new peak-SharePrice event in reinitialize, and add indexed to user in RequestSettled.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.7 DepositGatewayFactory::createDepositGateway has a dead gateway == address(0) check after new

Description: DepositGatewayFactory::createDepositGateway checks if (gateway == address(0)) revert FailedDeployment(...) immediately after gateway = address(new DepositGateway(...)). The new operator reverts the whole transaction if creation fails (out-of-gas, constructor revert, etc.) - it can never return address(0). The branch is unreachable dead code; the FailedDeployment error and its string argument can never fire from this site.

```
factory/DepositGatewayFactory.sol
18:     gateway = address(new DepositGateway(strategy_));
19:     if (gateway == address(0)) revert FailedDeployment("zero gateway address");
```

Recommended Mitigation: Remove the unreachable check, and remove the FailedDeployment import if it is no longer used. Note that the sibling V1 factories use the same dead pattern against new-created addresses; those are out of scope here but share the root cause.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.8 DepositGateway::_passEpoch passes an emptied epoch into Passed where it never finalizes to Settled

Description: cancelDeposit decrements epoch.pendingCount but does not change epoch.status or currentOpenEpochId (src/modules/DepositGateway.sol:113-132), so when every request in the open epoch is cancelled the epoch is left Open with pendingCount == 0. _passEpoch then closes such an epoch to Passed unconditionally - it never checks pendingCount (src/modules/DepositGateway.sol:269-277). The pendingCount == 0 finalization that flips an epoch to Settled exists only inside settle (src/modules/DepositGateway.sol:186) and refund (src/modules/DepositGateway.sol:212), so an emptied epoch passed by onNavUpdate (or force-passed via _openEpochIfNeeded or forcePassEpoch) lands in Passed with no pending requests and no path that auto-finalizes it. It emits EpochPassed for an epoch holding no live deposits and never emits EpochSettled, lingering in Passed indefinitely unless someone calls settle(epochId, new uint256[] (0)), whose post-loop pendingCount == 0 check then flips it to Settled.

No funds or liveness are affected: the escrow was already returned on cancel, unsettledEscrow was decremented, and currentOpenEpochId is zeroed at pass so the next request opens a fresh epoch. The impact is observational. Off-chain epoch-lifecycle trackers that expect every Passed epoch to eventually emit EpochSettled see the empty epoch stuck in Passed forever, and settle-scanning keepers keep surfacing it as outstanding. It is

reachable in normal operation: a sole subscriber who deposits then cancels (a cost-free action) before the NAV leaves the epoch empty when it passes.

Files:

- `DepositGateway::_passEpoch` (`src/modules/DepositGateway.sol`)
- `DepositGateway::cancelDeposit` (`src/modules/DepositGateway.sol`)

Recommended Mitigation: Finalize empty epochs at pass time. In `_passEpoch`, when `epoch.pendingCount == 0`, set `epoch.status = EpochStatus.Settled` and emit `EpochSettled` instead of leaving the epoch `Passed`, or skip passing an epoch that has no pending requests. Either keeps the epoch lifecycle observable off-chain and removes the lingering empty `Passed` state.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.9 `DepositGateway` wraps vault `IAccess` reads in redundant `try/catch` guards that cannot fail for a valid vault

Description: The gateway binds its vault once at construction (immutable) and the constructor already calls `IAccess(vault_).permissionLevel()` unwrapped (`src/modules/DepositGateway.sol:71`), so any deployed gateway is guaranteed to sit on a vault that implements `IAccess`. Yet three helpers still wrap their vault reads in `try/catch`: `_requireWhitelistIfConfigured` (`src/modules/DepositGateway.sol:319-335`), `_bothAllowed` (`src/modules/DepositGateway.sol:304-316`), and `_refundable` (`src/modules/DepositGateway.sol:291-296`). `permissionLevel` and `allowed` are plain storage and mapping getters; a `permissionLevel` call cannot fail against a valid vault, and `allowed` is a simple `mapping(address => bool)` lookup that likewise cannot revert, so every catch arm is unreachable code that only obscures the control flow.

The catch directions are also inconsistent: `_requireWhitelistIfConfigured`'s `permissionLevel` catch does a bare return (fail-open - silently skipping whitelist enforcement) while the `allowed` catches return false (fail-closed). None of this affects fund safety, since the vault's settle-time `onlyAuth / _areVerified` is the authoritative permission gate and these are only early-rejection checks, so it is a readability simplification with the bonus of removing the lone fail-open arm.

Files:

- `DepositGateway::_requireWhitelistIfConfigured` (`src/modules/DepositGateway.sol`)
- `DepositGateway::_bothAllowed` (`src/modules/DepositGateway.sol`)
- `DepositGateway::_refundable` (`src/modules/DepositGateway.sol`)

Recommended Mitigation: Call `permissionLevel` and `allowed` directly in all three helpers and drop the `try/catch` wrappers. A vault that cannot answer these getters is not a valid deployment target, so a clean revert is safer and simpler than silently skipping the check or special-casing it.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.3.10 Redemptions are priced at the live NAV with no front-running protection, unlike deposits, letting a holder exit ahead of a NAV-loss publish

Description: The protocol batches deposits through `DepositGateway` so they "enter `AccountableYield` only at the post-NAV price": a depositor commits capital before the NAV that prices them is known and therefore cannot front-run a NAV move. Redemptions have no equivalent guard. `onRequestRedeem` prices a redemption at the live `_sharePrice` and instant-fulfills it in the same transaction whenever liquidity is available and the NAV is not stale:

```
function onRequestRedeem(address share, uint256 shares, address, address)
    public override(AccountableStrategy, IStrategyVaultHooks) nonReentrant onlyVault whenNotPaused
    returns (bool canFulfill, uint256 price)
{
```

```

    if (shares < _loan.minRedeem) revert InsufficientShares();
    _accruePenalties();
    _accrueFees();
    price = _sharePrice(share);
    uint256 assets = shares.mulDiv(price, PRECISION);
    uint256 liquidity = _getAvailableLiquidity();
    canFulfill = liquidity >= assets && !_navIsStale();
}

```

`_navIsStale()` is false for the entire `navGracePeriod` (default 24h) after the last publish, so a redemption is priced at the last published NAV right up until the next `publishRate` overwrites it:

```

function _navIsStale() internal view returns (bool) {
    return block.timestamp > navGraceDeadline;
}

```

`publishRate` is a single, mempool-visible transaction whose `newDeployedValue` fully determines the new (lower) share price:

```

function publishRate(uint256 newDeployedValue, uint256 measuredAt) external {
    if (msg.sender != dvnPublisher) revert Unauthorized();
    _requireLoanOngoing();
    // ...
    deployedAssets = newDeployedValue;
    // ...
}

```

A holder who sees a pending loss publish (a `publishRate` with a lower `newDeployedValue` sitting in the mempool, or a foreseeable delinquent-borrower NAV drop) can `requestRedeem` first; with vault liquidity available the request instant-fulfills at the current pre-loss price, and the published loss then falls entirely on the remaining holders. This is the exit-side mirror of the front-running the deposit gateway exists to prevent on the entry side.

Impact: A holder who front-runs a NAV-loss publish redeems at the stale pre-loss price while vault liquidity lasts, extracting value from and shifting the marked-down loss onto the remaining holders, the exact attack the deposit gateway blocks on entry but which has no equivalent guard on exit.

Proof of Concept:

1. Vault has 1,000,000 shares, `deployedAssets` = 800,000, idle vault liquidity 300,000, share price 1.10. The NAV is about to be marked down to 600,000 (fair price 0.90).
2. The `dvnPublisher` broadcasts `publishRate(600_000, ...)`; the transaction sits in the mempool.
3. Eve front-runs it with `requestRedeem(272_727 shares)`. `onRequestRedeem` prices at the live 1.10 so `assets` = 300_000; `liquidity (300_000) >= 300_000` and the NAV is not yet stale, so `canFulfill` = `true` and the request is instant-fulfilled.
4. Eve receives 300,000 assets for shares worth 245,454 at the fair post-loss price.
5. `publishRate(600_000)` lands; the ~54,545 shortfall is absorbed by the remaining 727,273 holders.

Recommended Mitigation: Apply the same post-NAV pricing discipline to the exit side that the gateway provides on entry: route instant redemptions through a batch/queue that prices at the next published NAV rather than the last one, or block instant fulfillment within a window around a pending NAV update. At a minimum, bound the exposure with a shorter `navGracePeriod` and per-window liquidity caps so the amount that can exit at a stale price before a loss is published is limited.

Accountable: Fixed in commit [0f7abda](#) — gates the permissionless `accrueAndProcess` on `instantRedeemMaxAge`.

Cyfrin: The `maxAge` gate now covers both the instant path and the permissionless `accrueAndProcess` (which skips processing once the NAV is older than `maxAge`), so a redeemer can no longer `requestRedeem` then `accrueAndProcess` to settle at the stale NAV. Two residuals are accepted as Informational since impact is bounded by available liquidity: the borrower's repay still processes the queue ungated, and disabled mode (`maxAge` =

`type(uint256).max)` leaves `accrueAndProcess` ungated, so protection holds only in windowed mode ($0 < \text{maxAge} < \text{navGracePeriod}$)

7.3.11 `AccountableYield::_accruePenalties` includes accruedPenalties in the penalty base, charging compound interest instead of simple interest

Description: Each penalty increment is computed as a fraction of `_totalAssets`:

```
uint256 penaltyAmount =
    _totalAssets(vault).mulDiv(lateInterestPenalty_ * penaltyTime, DAYS_1_SECONDS * BASIS_POINTS);
...
accruedPenalties += penaltyAmount;
```

and `_totalAssets` includes `accruedPenalties`:

```
return IAccountableVault(vault_).totalAssets() + deployedAssets + accruedPenalties;
```

Because the base includes already-accrued penalties, each call charges interest on top of prior penalties, i.e. compound interest rather than simple interest on the outstanding debt. `_accruePenalties` runs frequently: on borrow, repay, `publishRate`, `updateTerms`, and on `updateLateStatus`, which the vault invokes after every deposit, mint, and redeem. The more often it runs, the closer the growth approaches continuous compounding. `lateInterestPenalty` has no upper bound in `setTerms` / `updateTerms`, so at high rates the divergence is large: at 10%/day over 30 days, simple interest is 300% of the base while continuous compounding is roughly 1,900%.

Impact: Penalties grow super-linearly with time and accrual frequency, inflating `accruedPenalties`, `_totalAssets`, and the share price beyond the intended simple-interest schedule, so depositors entering at the inflated price are mispriced and absorb the collapse when the borrower defaults and the phantom penalties are written off.

Recommended Mitigation: Compute the penalty increment from a base that excludes prior penalties, so accrual is simple interest on the principal:

```
+ // Simple interest on the principal base, excluding already-accrued penalties
+ uint256 penaltyBase = IAccountableVault(vault).totalAssets() + deployedAssets;
+ uint256 penaltyAmount =
-     _totalAssets(vault).mulDiv(lateInterestPenalty_ * penaltyTime, DAYS_1_SECONDS * BASIS_POINTS);
+     penaltyBase.mulDiv(lateInterestPenalty_ * penaltyTime, DAYS_1_SECONDS * BASIS_POINTS);
```

Accountable: Fixed in commit [c54cb88](#)

Cyfrin: Verified. The penalty base now excludes `accruedPenalties`, so penalties accrue as simple interest rather than compounding.

7.4 Gas Optimization

7.4.1 DepositGateway::_passEpoch re-derives the epochs[epochId] storage pointer the callers already hold

Description: DepositGateway::onNavUpdate and forcePassEpoch already hold (or compute) the epochs[epochId] storage slot, then call _passEpoch(epochId, ...), which re-hashes keccak256(epochId . slot) to re-derive the same Epoch storage pointer. With the optimizer off, the pointer derivation and the status read are not deduped across the call boundary. Passing the already-derived Epoch storage pointer (or caching it) avoids one mapping-slot re-derivation plus the extra cold/warm SLOAD per close. This is a low-frequency path (NAV publish / recovery), so the savings are modest, but the dedup is guaranteed under optimizer-off.

```
DepositGateway.sol
139:         if (epochs[epochId].status != EpochStatus.Open) return; // unexpected - no-op, never revert
141:         uint256 nav = _passEpoch(epochId, IMinimalStrategy(strategy));

153:         if (strategy_.lastNavMeasuredAt() <= epochs[epochId].openNavMeasuredAt) {
157:             uint256 nav = _passEpoch(epochId, strategy_);

269:         function _passEpoch(uint256 epochId, IMinimalStrategy strategy_) private returns (uint256
    ↪ navMeasuredAt) {
270:             Epoch storage epoch = epochs[epochId];
```

Recommended Mitigation: Change _passEpoch to take an Epoch storage epoch parameter (the epochId is otherwise needed only for the lookup), and have each caller pass the pointer it already read. For example, in onNavUpdate cache Epoch storage epoch = epochs[epochId]; once, gate on epoch.status, and call _passEpoch(epoch, ...).

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.4.2 DepositGateway::_passEpoch re-reads strategy NAV every caller already fetched

Description: DepositGateway::_passEpoch calls strategy_.lastNavMeasuredAt() again even though every caller has just read it: onNavUpdate (via the status path), forcePassEpoch at line 153, and _openEpochIfNeeded at lines 249/263 all read lastNavMeasuredAt immediately before invoking _passEpoch, with no intervening write. The re-read inside _passEpoch issues a redundant cross-contract STATICCALL on the epoch-close path.

```
DepositGateway.sol
153:         if (strategy_.lastNavMeasuredAt() <= epochs[epochId].openNavMeasuredAt) {
157:             uint256 nav = _passEpoch(epochId, strategy_);
249:             if (strategy_.lastNavMeasuredAt() <= epochs[epochId].openNavMeasuredAt) {
263:                 epoch.openNavMeasuredAt = strategy_.lastNavMeasuredAt();
271:                 navMeasuredAt = strategy_.lastNavMeasuredAt();
```

Recommended Mitigation: Change _passEpoch(uint256 epochId, IMinimalStrategy strategy_) to accept the NAV value the caller already holds (e.g. _passEpoch(uint256 epochId, uint256 navMeasuredAt)), removing the redundant lastNavMeasuredAt STATICCALL.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.

7.4.3 AccountableYield::_accrueFees re-reads the vault totalSupply already loaded by _accruedFeeShares

Description: _accrueFees runs on every deposit, mint, borrow, repay, and NAV publish. It calls _accruedFeeShares (src/strategies/AccountableYield.sol:504), which reads the vault totalSupply (src/strategies/AccountableYield.sol:565), and then _accrueFees reads totalSupply again (src/strategies/AccountableYield.sol:511) to compute the post-fee high-water mark. No shares are minted

between the two reads (the fee-share mints happen later at `src/strategies/AccountableYield.sol:545,549`), so the second cross-contract `STATICCALL` returns an identical value and repeats work already done. `_sharePrice` (`src/strategies/AccountableYield.sol:617-623`) has the same pattern: it calls `_accruedFeeShares` and then re-reads `totalSupply`.

```
AccountableYield.sol
504:      (uint256 performanceFeeShares, uint256 managementFeeShares, uint256 newTotalAssets) =
↳ _accruedFeeShares();
565:      uint256 supply = IAccountableVault(vault_).totalSupply();    // inside _accruedFeeShares
511:      uint256 totalSupply = IAccountableVault(vault).totalSupply(); // re-read on the same path,
↳ no mint between
```

Recommended Mitigation: Have `_accruedFeeShares` also return the supply it already loads, and consume that value in `_accrueFees` (and in `_sharePrice`) instead of issuing a second `totalSupply` call. This removes one external `STATICCALL` per fee accrual on every deposit, mint, borrow, repay, and NAV-publish path.

Accountable: Fixed in commit [aea937c](#)

Cyfrin: Verified.